

4



Input

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{bmatrix}$$

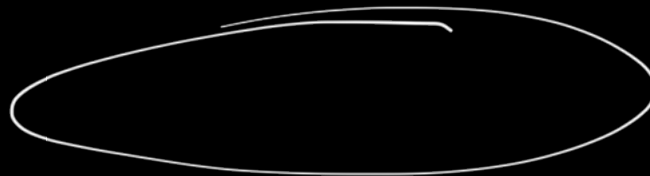
Dette er et fire tall!

SOK-3023 Maskinl ring for  konomer

Beslutningstr r og Random Forests

Prediksj n, fleksibilitet og ensemble-metoder

Lin. reg.



NN

Kompleksitet

Agenda

1. Hva er beslutningstrær?
2. Hvordan bygges et tre?
3. Overfitting og pruning 
4. Fra enkelttrær til Random Forests
5. Bias-Variance Trade-off i praksis

Mål: Forstå hvorfor mange "dårlige" modeller kan bli én veldig god.

"weak learners"

— " —
— " —
— " —

God til å
predikere.

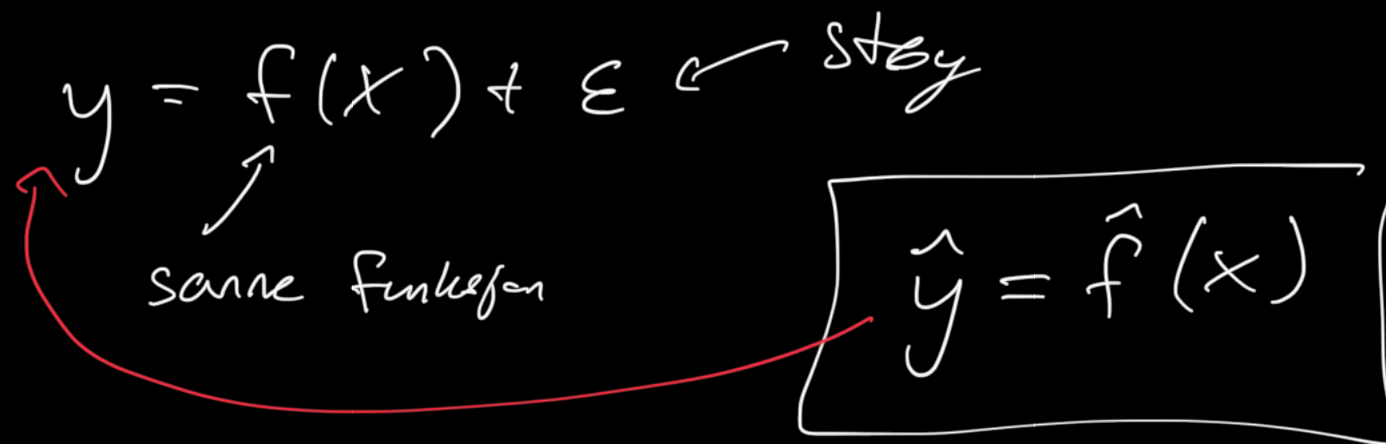
Veiledet læring

Vi gir datamaskinen/modell masse eksempler, med en fasit - og ønsker at modellen skal lære seg strukturen i dataene.

$$y = f(x) + \epsilon \quad \leftarrow \text{støy}$$

↑
sanne funksjon

$\hat{y} = \hat{f}(x)$



Mål:

$\hat{f}(x)$ skal komme så nær $f(x)$ som mulig.

Feilen skal bli så liten som mulig.

$$r = y - \hat{y} = \boxed{f(x) - \hat{f}(x)} + \epsilon$$

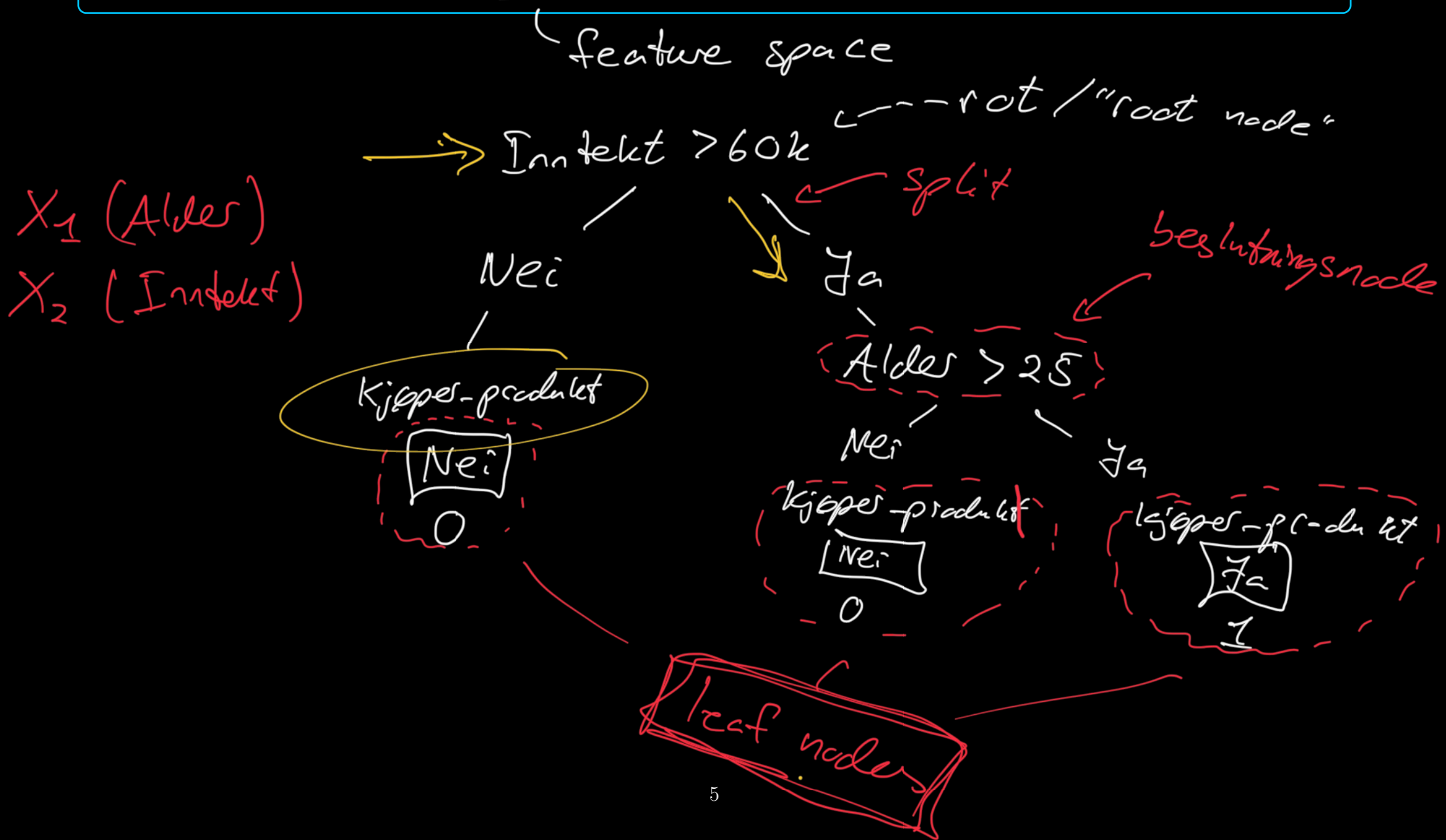
Eksempel

Vi tenker at vi har et datasett som handler om Alder (X_1), Inntekt (X_2) og Kjøper_Produkt (y)

Features		Target
Alder (X_1)	Inntekt (X_2)	Kjøper-produkt (y)
22	30000,-	Nei ← 0
35	80000,-	Ja ← 1
45	60000,-	Ja ← 1
⋮	⋮	⋮

Hva er et beslutningstre?

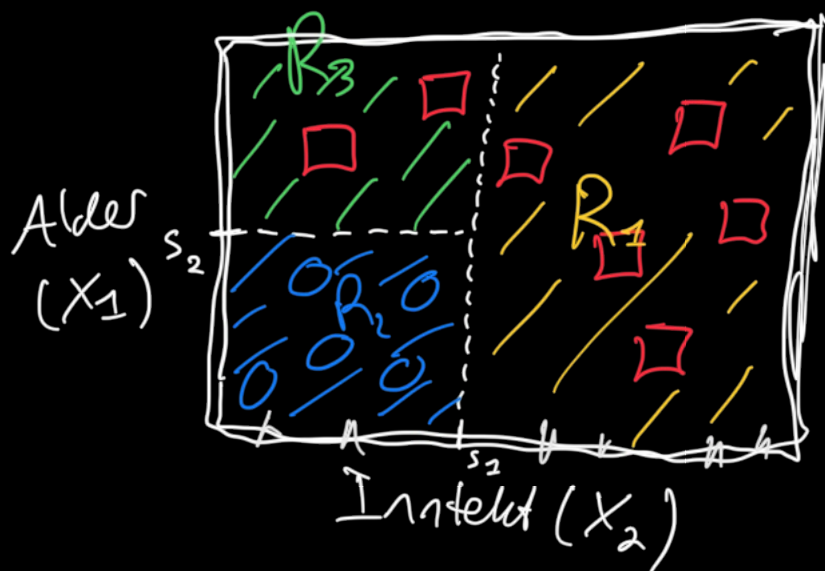
Et beslutningstre er en modell som deler opp datarommet i regioner ved hjelp av **sekvensielle ja/nei-spørsmål**.



Hva er et beslutningstre?

Et beslutningstre er en modell som delar opp datarommet i regioner ved hjelp av sekvensielle ja/nei-spørsmål.

Feature space



$$R_1 = \{X \mid \text{Inntekt}(X_2) > s_1\}$$

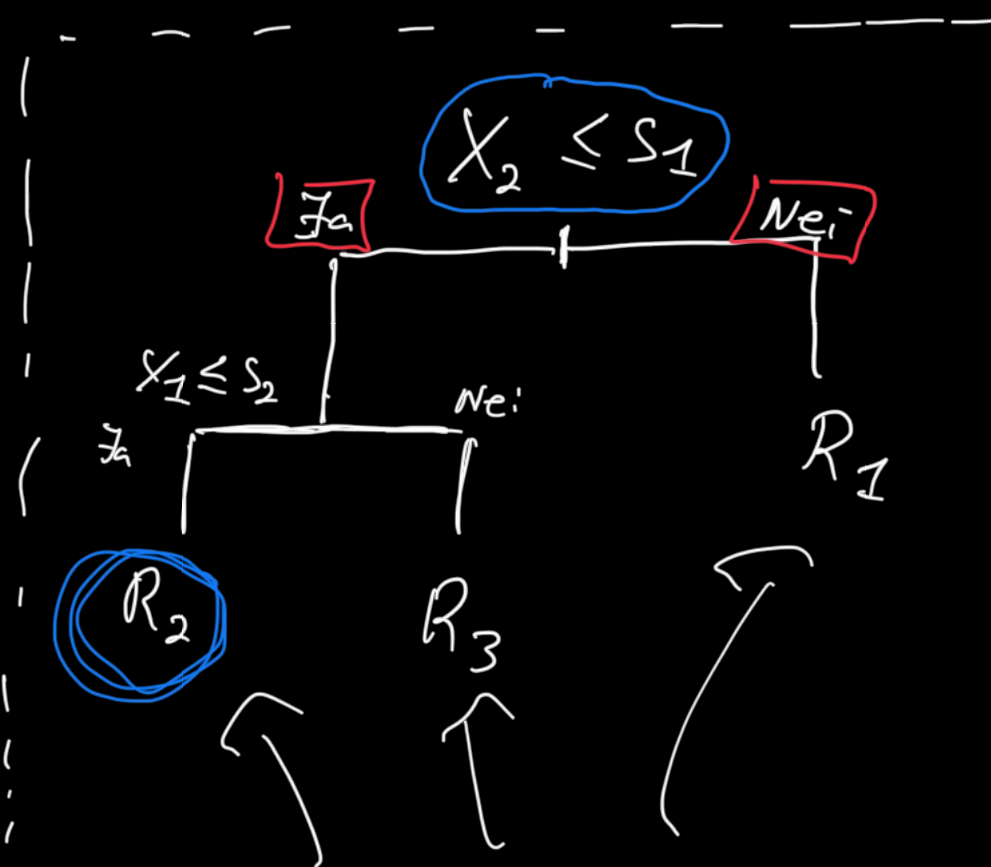
$$R_2 = \{X \mid \text{Inntekt}(X_2) \leq s_1, \text{Alder} \leq s_2\}$$

$$R_3 = \{X \mid \text{Inntekt}(X_2) \leq s_1, \text{Alder} > s_2\}$$

$\square$ = Kjøper-produkt ($\bar{J}_a$)

$\circ$ = — " — ($\bar{N}_e$)

Mål: Del feature space i ikke-overlappende regioner.



Hvordan splittes faktisk trær?

Da brukes noe som kalles binary recursive splitting.

Binary $\rightarrow$ hver split deler en node i to grener (50/50%).

Recursive $\rightarrow$ prosessen gjentas på hver nye node.

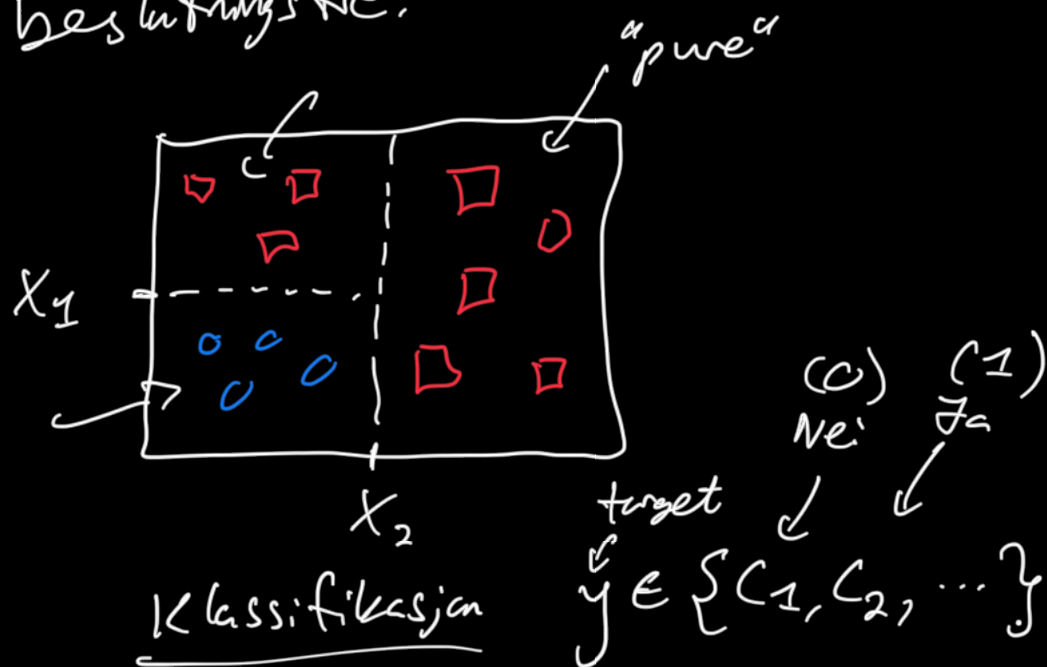
Kort forklart

1. Starter med hele datasettet i roten
2. Prøver alle mulige splitter på alle features ($x_1, x_2, x_3, \dots$)
3. Velger der splitter som minimerer impurity
"urenhet"
4. Del noder i to nye noder.
5. Gjenta prosessen til treet stopper.
maks. dybde
min. samples

Hvordan splittes faktisk trær?

Da brukes noe som kalles *binary recursive splitting*.

Impurity er et mål på hvor "blandet" en node er i et beslutnings tre.

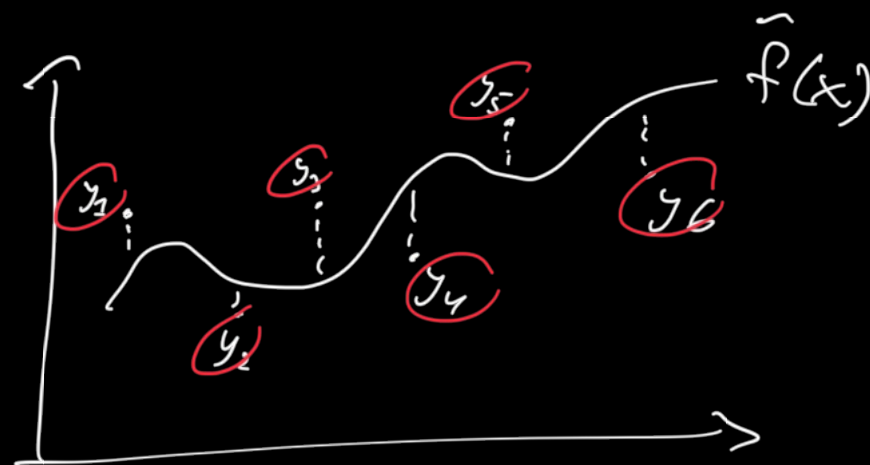


- Gini - index

Regression y ∈ R

Hvor spredt er verdiene.

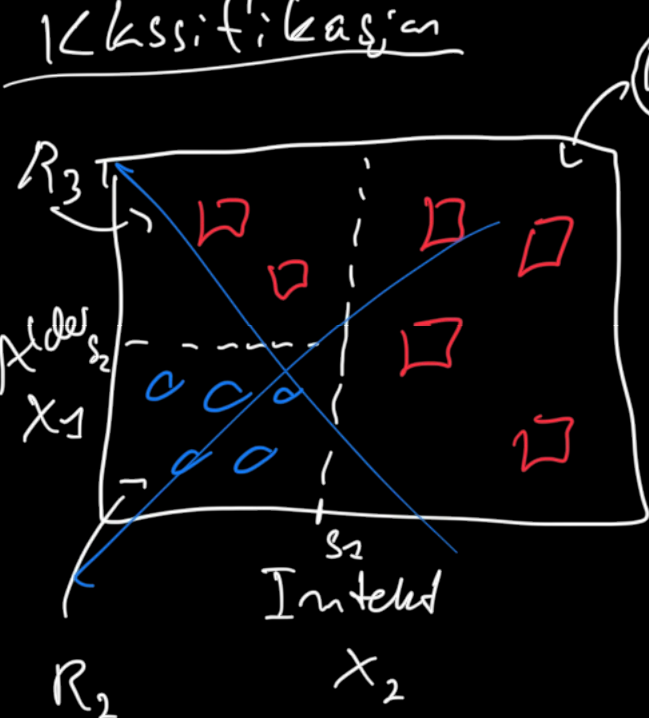
$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$



Hvordan splittes faktisk trær?

Da brukes noe som kalles *binary recursive splitting*.

1-klassifisering



$\square = \text{Kjøper (Ja)}$
 $\circ = \text{Nei (Nei)}$

Eks.

Node	Ja	Nei
R1	8	0
R2	1	8
R3	3	0

Gini for R1:

$$Gini = 1 - \sum_{i=1}^2 p_i^2$$

$$= 1 - \left(\left(\frac{8}{8}\right)^2 + \left(\frac{0}{8}\right)^2\right)$$

$$= 1 - 1 = \underline{0} \text{ Perfekt!}$$

Gini-index:

$$Gini = 1 - \sum p_i^2$$

p_i = andel av klasse i noder.

$Gini = 0 \rightarrow$ helt ren "pure" node

"Makes gini" for 2 klasser er 0.5

Gini for R2:

$$Gini = 1 - \left(\left(\frac{1}{9}\right)^2 + \left(\frac{8}{9}\right)^2\right) \approx 0.198$$

"ganske ren"

Trær for klassifikasjon

Når Y er kategorisk, bruker vi ikke RSS.

MSE

regresjon.

Vanlige mål:

- Gini-indeks

- Entropi

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

$$H = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

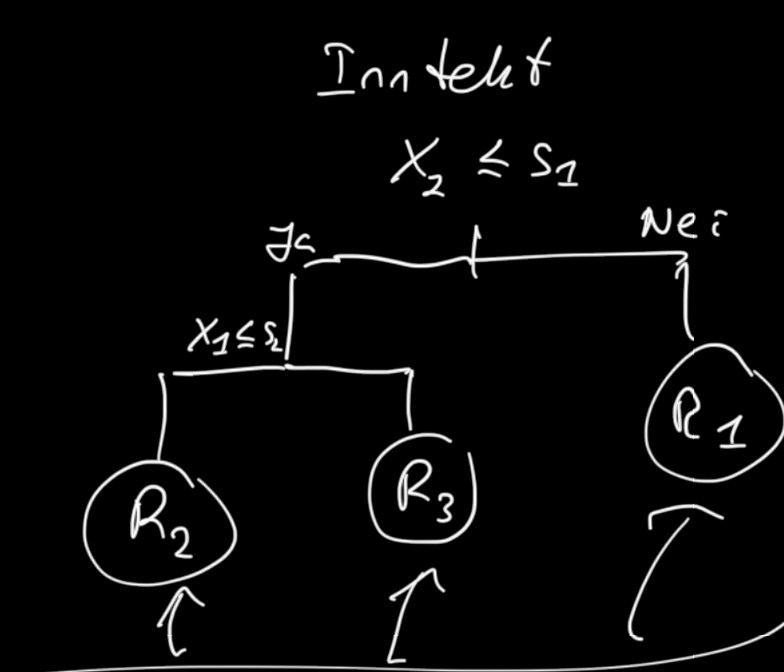
seminar oppgave

Lav verdi = rene noder

← ←

Trær for klassifikasjon

Mer om impurity



Hvordan impurity brukes

For hver mulige split

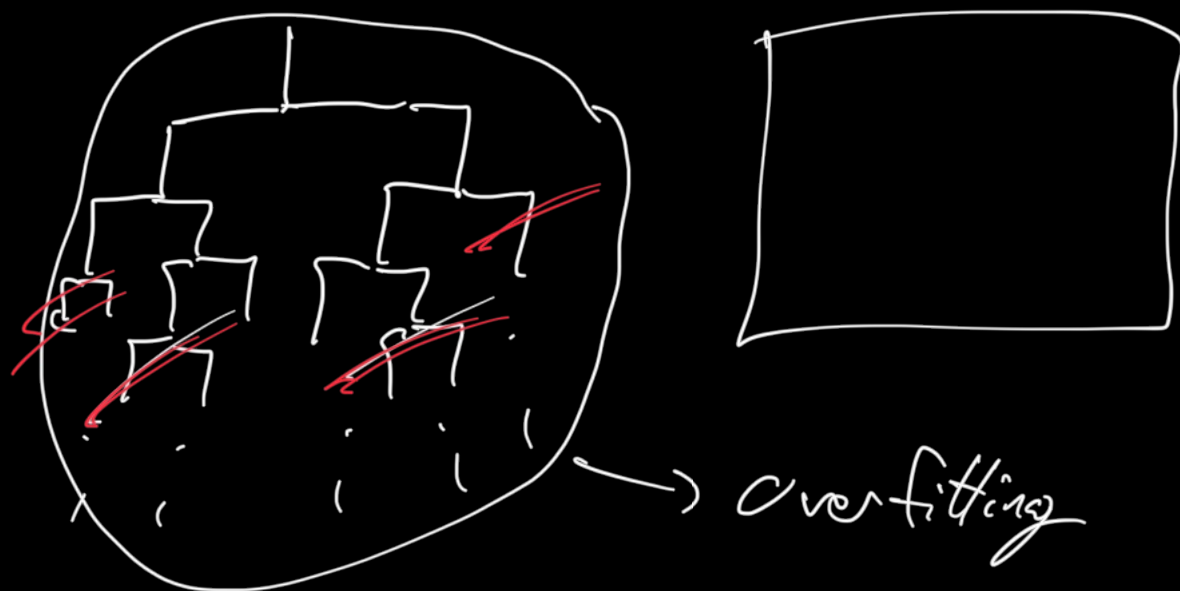
- (1.) Beregn impurity for split.
- (2.) — — — etter split.
- (3.) Velger vi den splitten som gir størst reduksjon i impurity.

Gini

MSE

Problemet med beslutningstrær

Teknikken er veldig sensitiv for endring i treningsdata.



kan ha lav S_{in}

Cost complexity penalty.

Mer "general"
↓
Mer robust til
større n , m og
test data.

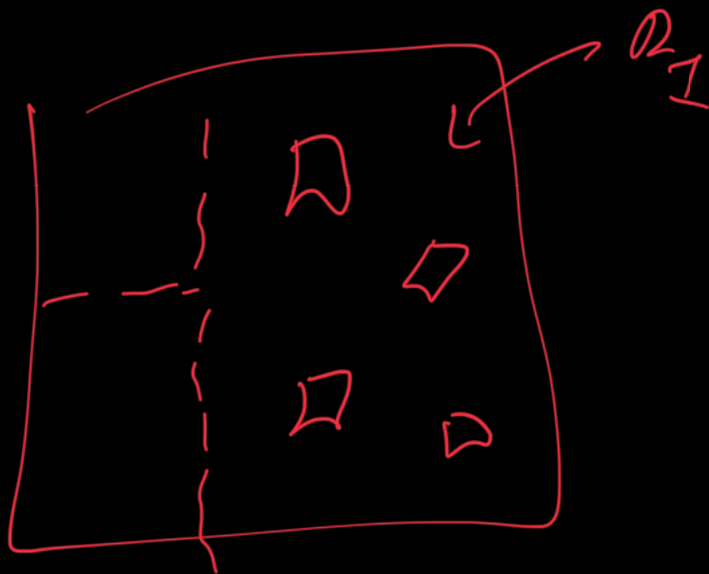
Beslutningstrær for regresjon

Ved regresjon predikerer hvert blad gjennomsnittet av observasjonene i regionen.

Modellen er:

$$\hat{f}(x) = \sum_{m=1}^M c_m \cdot I(x \in R_m)$$

- R_m : Regioner (rektangler i X -rommet)
- c_m : Gjennomsnittlig Y i region m



Hvordan velges splitt?

Vi velger splitt som **minimerer RSS** (Residual Sum of Squares).

$$\text{RSS} = \sum_{m=1}^M \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2$$

- Tester alle variabler X_j
- Tester alle terskler c
- Velger beste kombinasjon

Mer om i kompendium!

Problemet med beslutningstrær

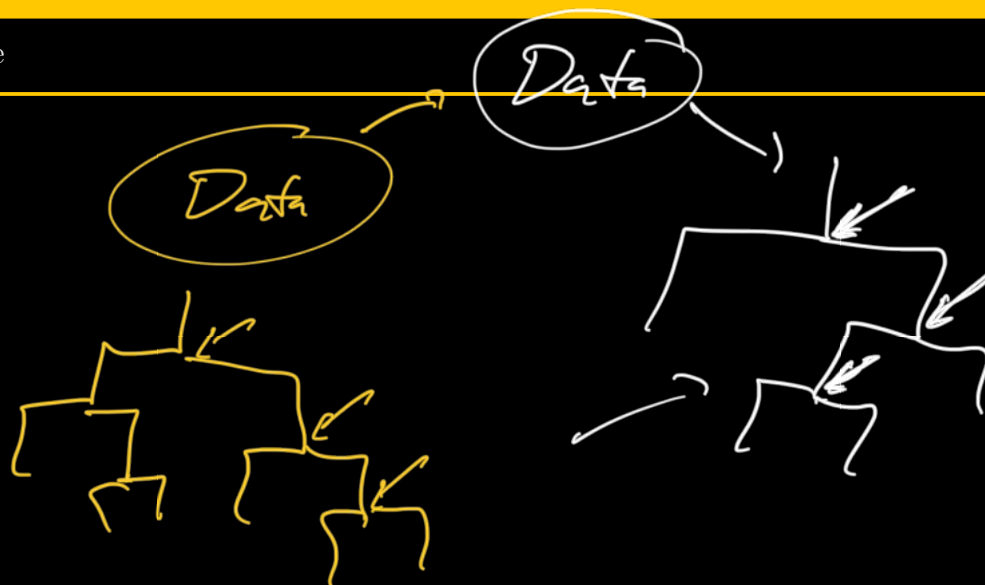
Et fullt utvikst tre har:

- Lav bias
- Ekstremt høy varians

Viktig: Konsekvens

Små endringer i dataene $\Rightarrow$ helt nytt tre

Løsning: Ikke la trær vokse fritt.

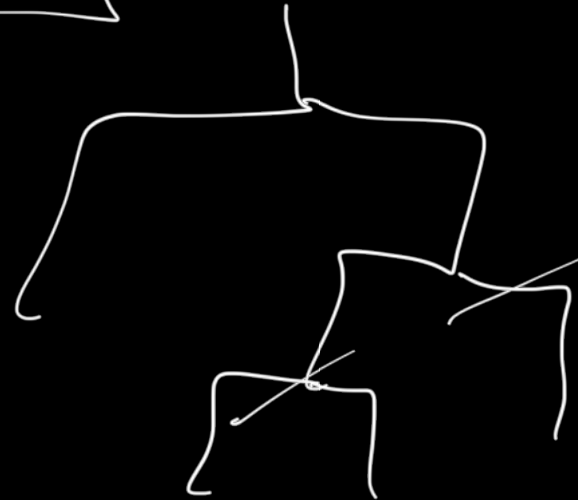


Pruning (beskjæring)

Vi straffer kompleksitet:

$$\sum_{m=1}^M \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|$$

- $|T|$: Antall noder
- α : Kompleksitetsstraff



Hvorfor ikke bare bruke mange trær?

Ett tre er ustabilt.

- Idé: Bygg mange trær
- La dem stemme (klassifikasjon)
- Eller ta gjennomsnitt (regresjon)

Dette er ensemble-metoder.



Modell

Lin. reg.

Best. tre

Bias

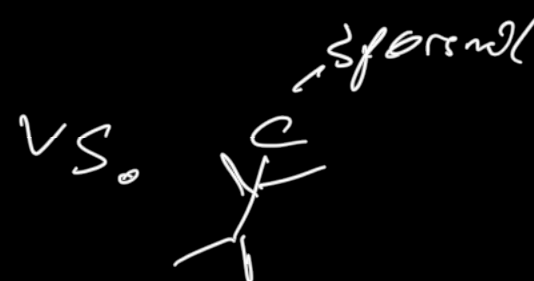
Høy /

Lav

Varians

Lav

Høy

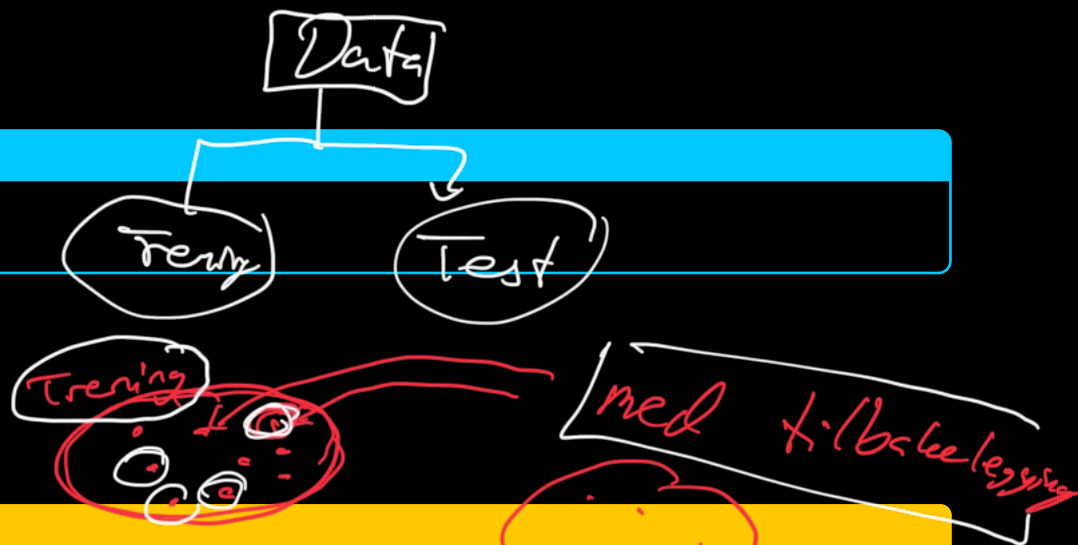


Random Forests

Random Forest – hovedidé

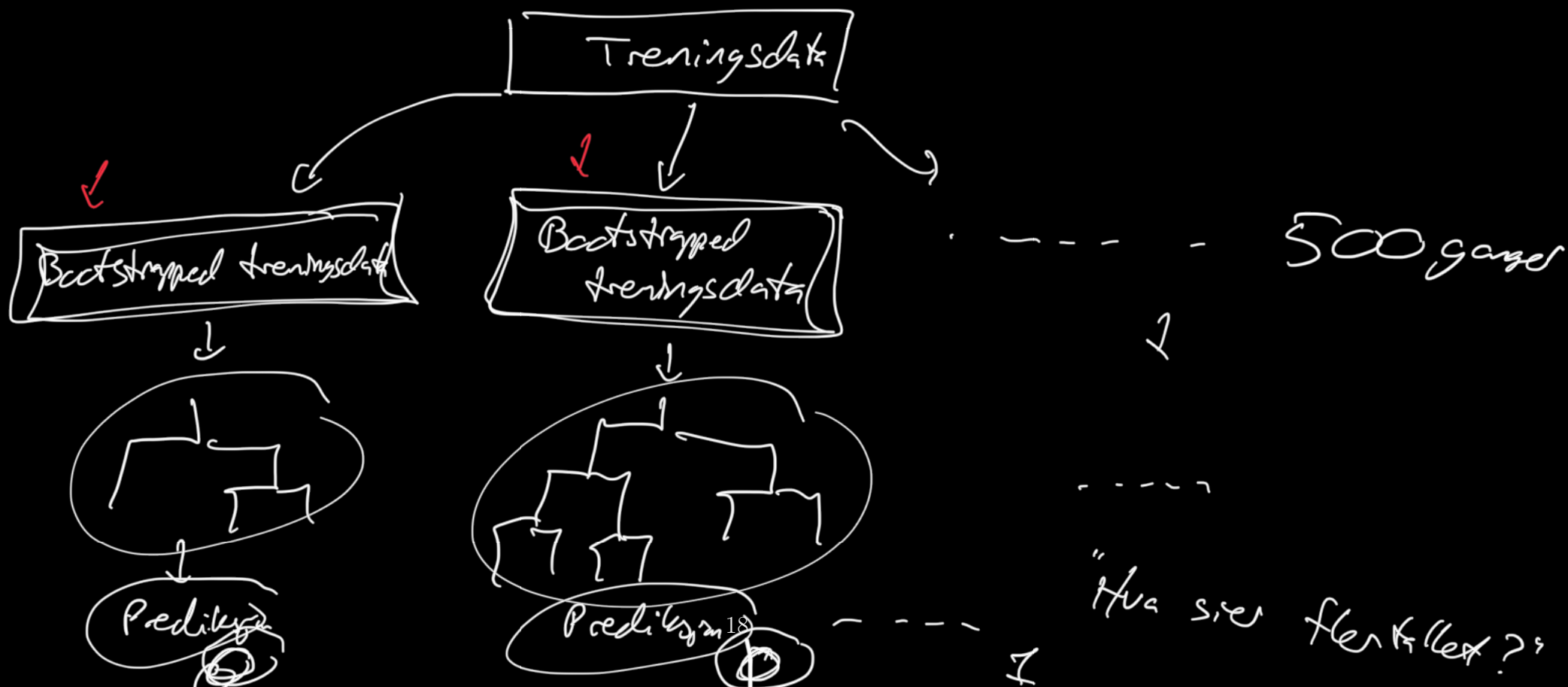
Random Forest kombinerer:

1. Bootstrap sampling (bagging)
2. Tilfeldig utvalg av variabler i hver splitt



Viktig: Nøkkelpoeng

Trærne blir mindre korrelerte $\Rightarrow$ lavere varians



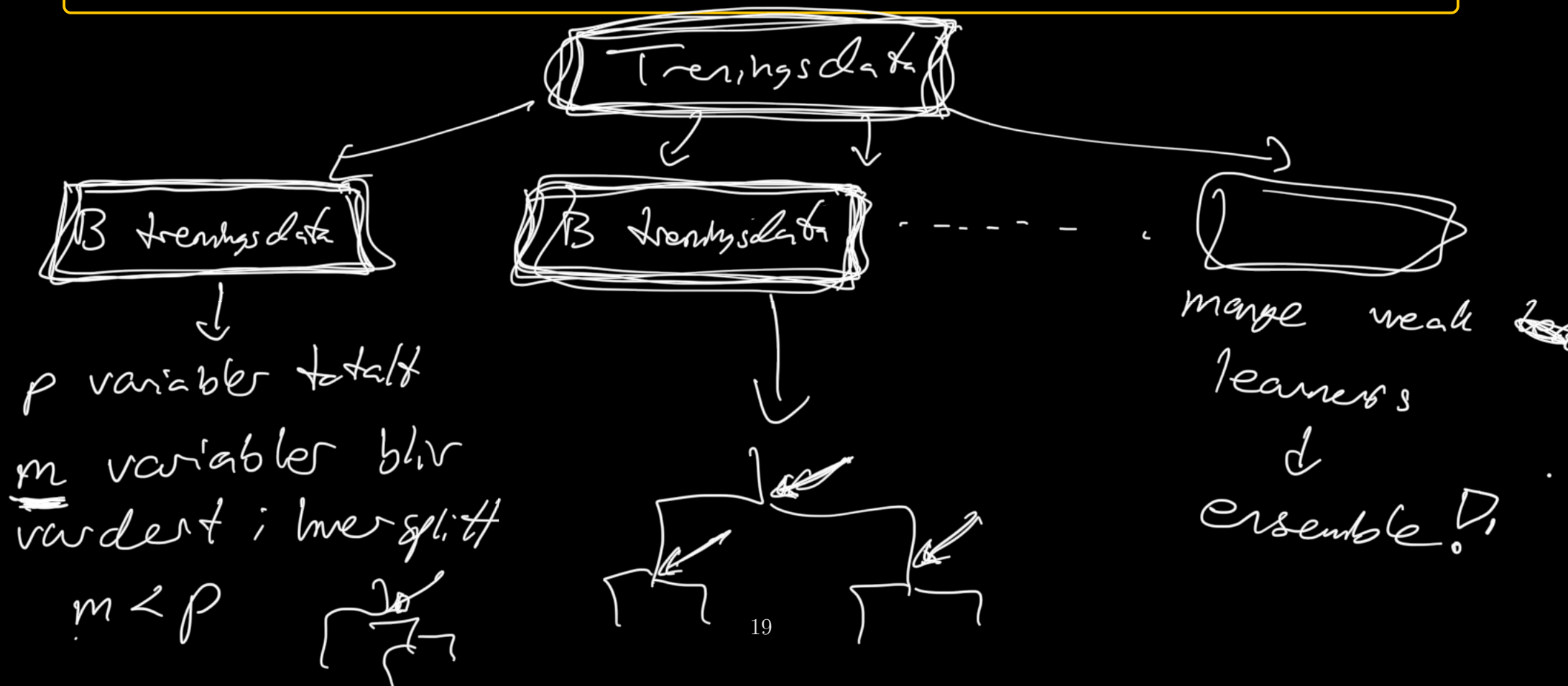
Random Forest – hovedidé

Random Forest kombinerer:

1. Bootstrap-sampling (bagging)
2. Tilfeldig utvalg av variabler i hver splitt

Viktig: Nøkkelpoeng

Trærne blir mindre korrelerte $\Rightarrow$ lavere varians



Random Forest-algoritmen

Input:

- Treningsdata: $\{(x_i, y_i)\}_{i=1}^n$
- Antall trær: B
- Antall features vurdert per splitt: $m < p$ (ofte $m = \sqrt{p}$)
- Stoppkriterier (maks dybde, min. samples, etc.)

x 'er (prediktorer, features)
 y er målvariabel, target
 $m = p$ (bagging)



Algoritme: For $b = 1, 2, \dots, B$:

1. **Bootstrapping:** Trekk et treningssett D_b ved tilfeldig sampling med tilbakelegging fra de originale dataene.
2. Bygg et tre $\hat{f}_b$ basert på D_b :
 - Start i roten og gjenta rekursivt for hver node:
 - (a) Velg tilfeldig m av de p tilgjengelige input-featurene.
 - (b) Finn den beste splitten blant de m featurene ved å minimere impurity:
 - **Klassifikasjon:** Gini-indeks / Entropy
 - **Regresjon:** RSS / MSE
 - (c) Del noden i to.
 - Stopp når et stoppkriterium er nådd.
3. Lagre treet $\hat{f}_b$.

$D_1, D_2, D_3, \dots, D_B$

Prediksjon for nytt datapunkt x :

Klassifikasjon:

$$\hat{y} = \text{maj. vote}\{\hat{f}_1(x), \dots, \hat{f}_B(x)\}$$

Regresjon:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$



Bias–Variance i Random Forests

Hvorfor gjør vi dette? Et enkelt beslutningstre er ofte dypt og komplekst, noe som gir lav bias, men høy varians (overfitting). Random Forest reduserer variansen gjennom to hovedmekanismer:

- **Bagging (Bootstrap Aggregating):** Ved å trene mange trær på ulike subsett av dataene og beregne gjennomsnittet, kanselleres tilfeldig støy. Variansen til et gjennomsnitt av n uavhengige modeller er $\frac{\sigma^2}{n}$. $\rightarrow$ variansen minsker! (ISLP)
- **Dekorrelering (Feature Randomness):** Ved å tvinge hvert splitt til kun å vurdere et tilfeldig utvalg av forklaringsvariabler, unngår vi at alle trærne blir like. Dette er kritisk fordi gjennomsnitt av korrelerte trær ikke reduserer variansen effektivt.
- **Trade-off:** Vi aksepterer en minimal økning i bias (fordi hvert tre begrenses) mot en massiv reduksjon i variens.

Resultat: En robust modell som generaliserer langt bedre på usette data enn et enkelt tre.

Prediksjon

Oppsummering

- Veldig god prediksjon
- Lite tuning
- Robust mot overfitting
- Men: Lite tolkbar

RF, krever mindre data enn NN

←

←

←

